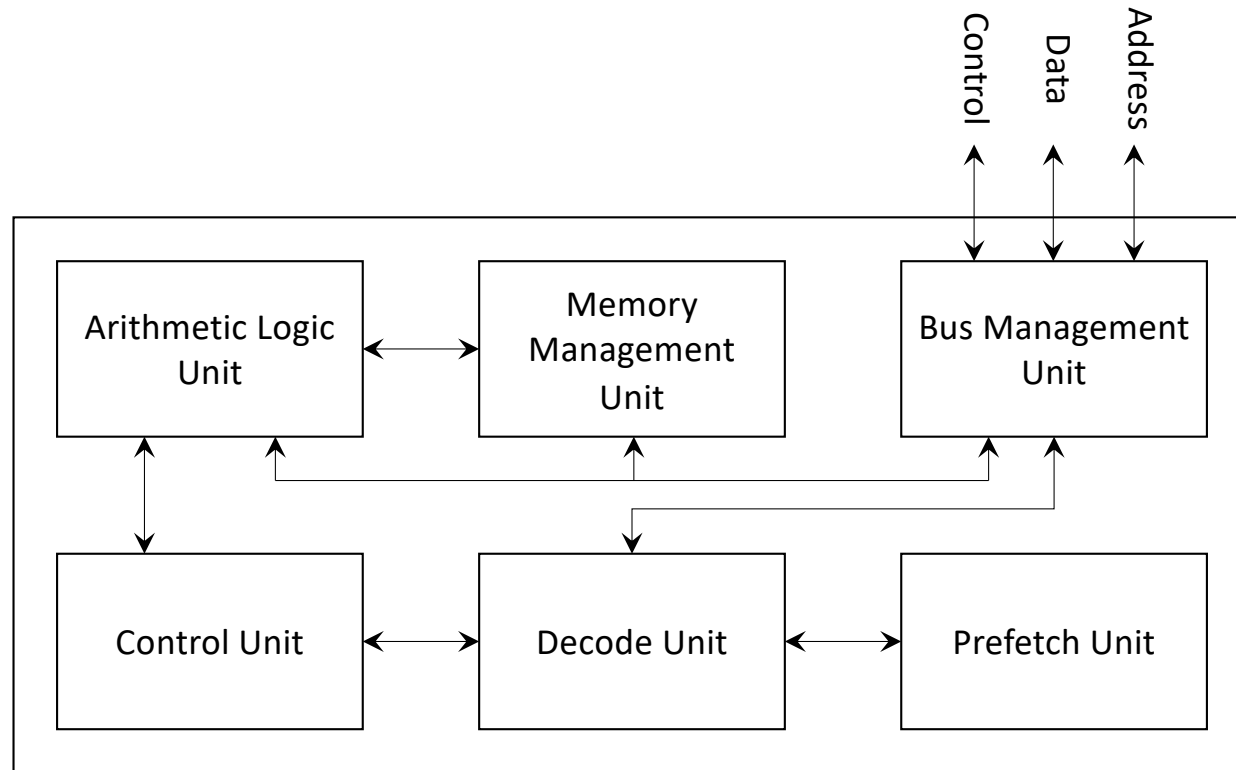


Microprocessor and Computer Architecture

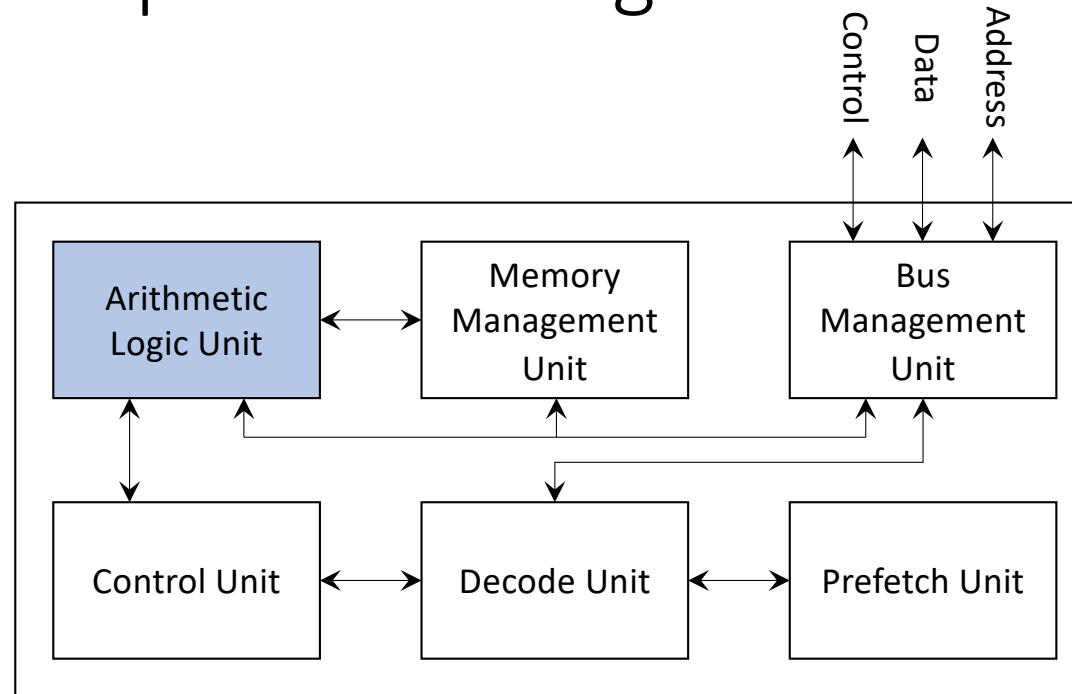
- ECSE 201L

L	T	P	Credits
3	1	2	4

Microprocessor Organization

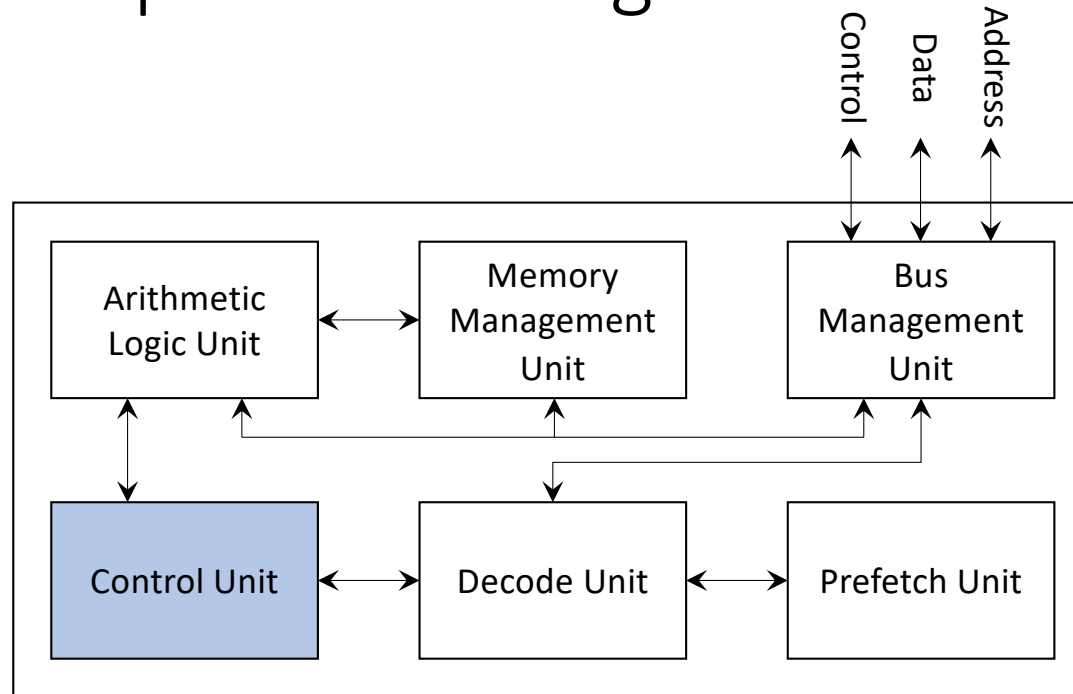


Microprocessor Organization



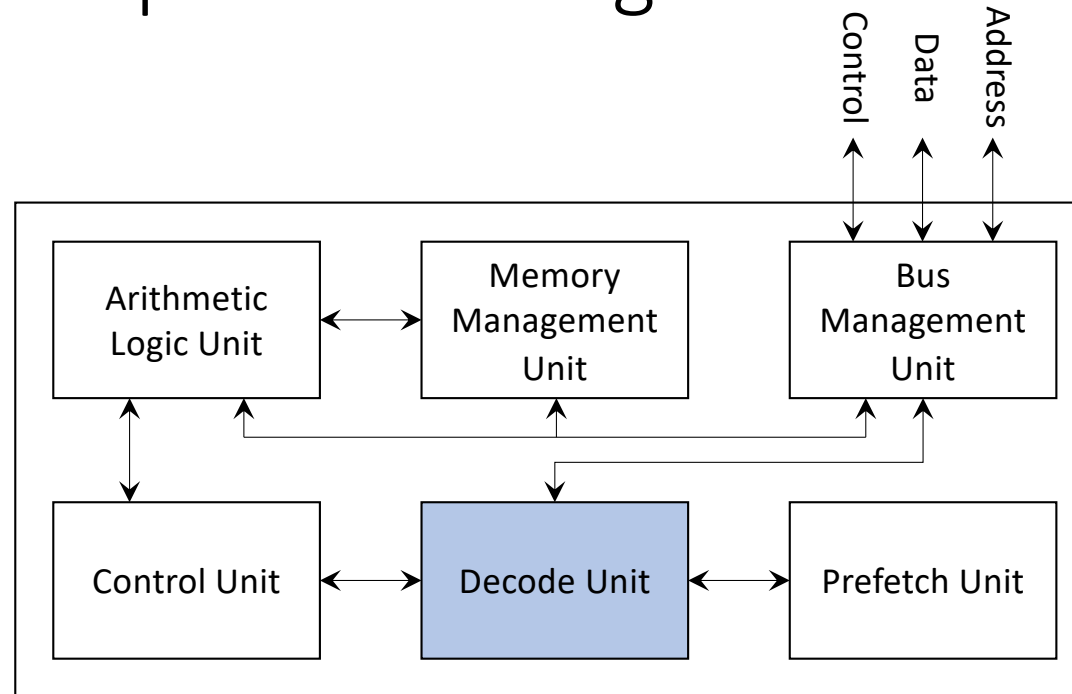
- **Arithmetic Logic Unit:** executes all logic and arithmetic operations as specified by the instruction set

Microprocessor Organization



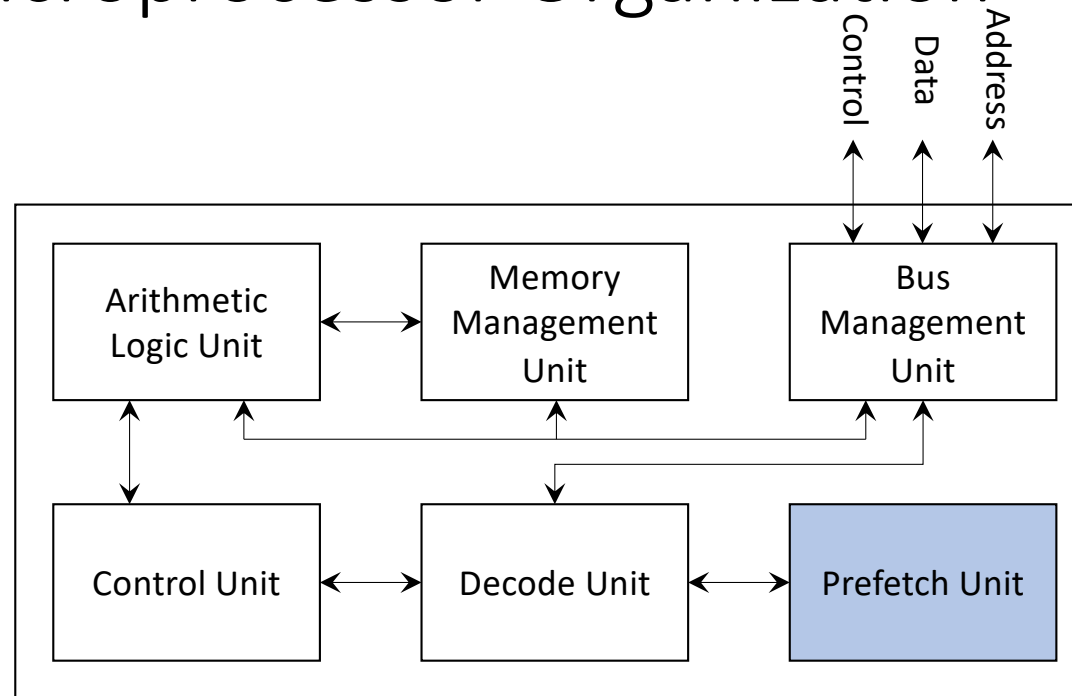
- Control Unit:** contains the microcode that tells the ALU how to function.

Microprocessor Organization



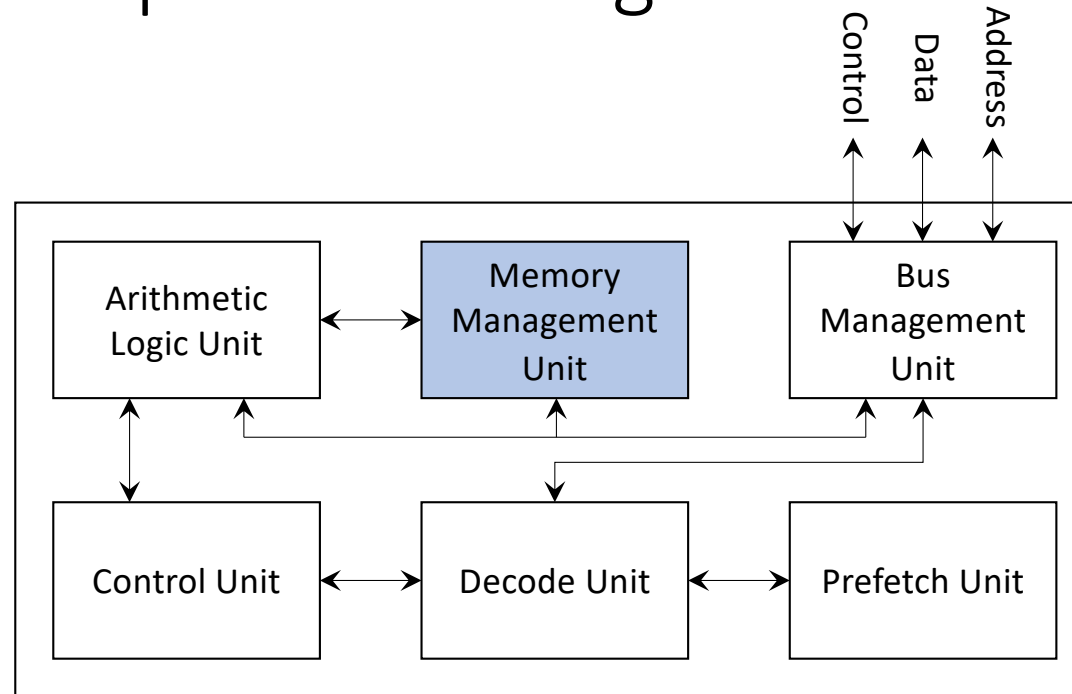
- **Decode Unit:** translates instruction into control signals and microcode directions then queues them until requested by the Control Unit.

Microprocessor Organization



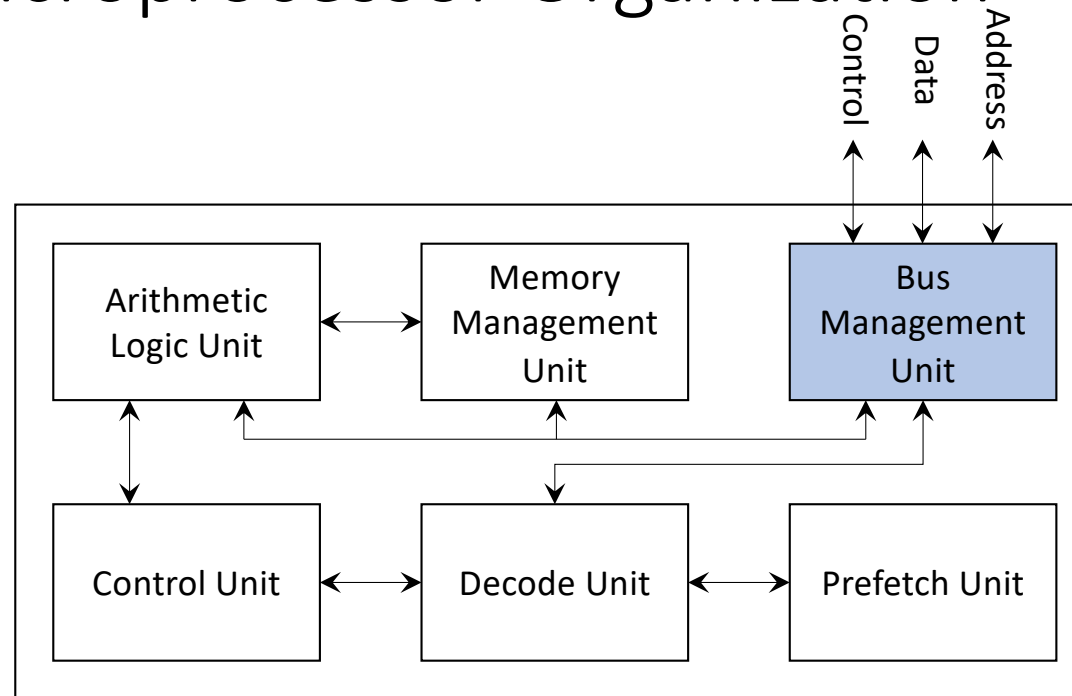
- Prefetch Unit:** queues instruction to assure that the microprocessor is in continuous operation.

Microprocessor Organization



- **Memory Management Unit:** converts internal logic addresses into external memory addresses.

Microprocessor Organization



- **Bus Management Unit:** manages the flow of information between the microprocessor and data storage locations (main disk, CD-ROM, etc.) and peripherals (printer, monitor, etc.)

Computer Arithmetic

Chapter Organization

- Representing negative numbers
- Integer addition and subtraction
- Integer multiplication and division
- Floating point operations

General concept

- Decimal addition

$$\begin{array}{r} \text{(carry)} \ 1 \\ 19 \\ + 7 \\ \hline 26 \end{array}$$

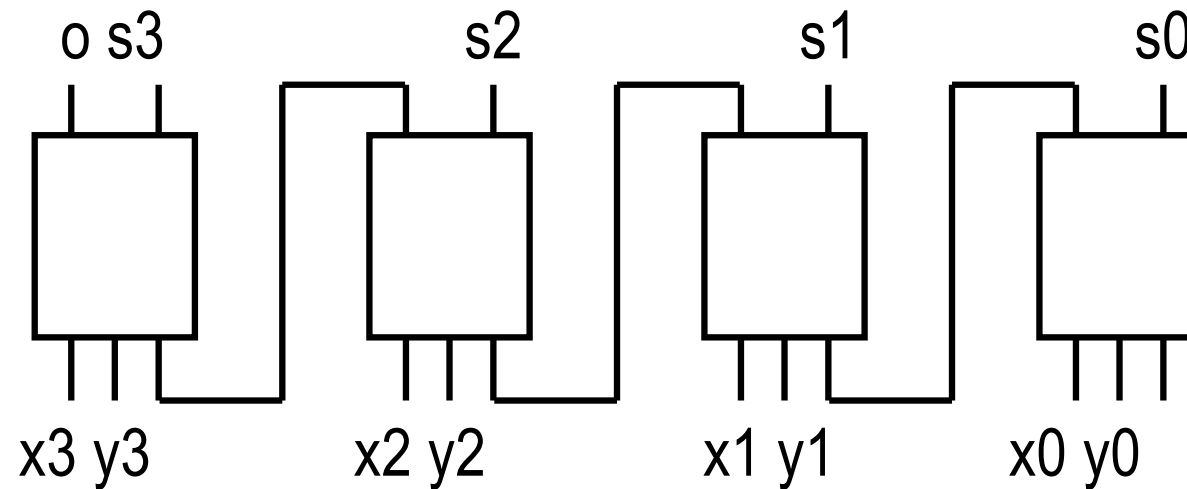
- Binary addition

$$\begin{array}{r} \text{(carry)} \ 111 \\ 10011 \\ + 111 \\ \hline 11010 \end{array}$$

- $16+8+2 = 26$

Realization with adders

- Simplest solution is a battery of full adders



Observations

- Adder add four-bit values
- Output **o** indicates if there is an ***overflow***
 - A result that cannot be represented using 4 bits happens when $x + y > 15$
- Operation is slowed down by ***carry propagation***
 - ***Faster solutions*** (not discussed here)

Signed and unsigned additions

- Unsigned addition in 4-bit arithmetic

$$\begin{array}{r} \text{(carry)} \quad 11_ \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

- $11 + 3 = 14$
 $(8 + 4 + 2)$

- Signed addition in 4-bit arithmetic

$$\begin{array}{r} \text{(carry)} \quad 11_ \\ 1011 \\ + 0011 \\ \hline 1110 \end{array}$$

- $-5 + 3 = -2$

Examples (using 5-bit signed numbers)

Case 1: (+7) + (+4)

+7=00111
+4=00100

00111+00100 = 01011
(Result: +11)

Case 2: (-7) + (-4)

-7=11001 (2's complement of 7)
-4=11100 (2's complement of 4)

11001+11100 = 110101

Discard carry-out:

Result: -11 (2's complement, Sign bit 1)

Case 3: (-7) + (+4)

-7=11001
+4=00100

00111 + 00100 = 11101 (-3 sign bit 1)

Signed and unsigned additions

- **Same rules** apply even though bit strings represent **different values**
- Sole difference is **overflow handling**

Overflow handling (I)

- No overflow in signed arithmetic

$$\begin{array}{r} \text{(carry)} \quad 111_ \\ \quad 1110 \\ + \quad 0011 \\ \hline \quad 0001 \end{array}$$

- $-2 + 3 = 1$
(correct)

- Signed addition in 4-bit arithmetic

$$\begin{array}{r} \text{(carry)} \quad 1_ \\ \quad 0110 \\ + \quad 0011 \\ \hline \quad 1001 \end{array}$$

- $6 + 3 \neq -7$
(false)

Overflow handling (II)

- In signed arithmetic an overflow happens when
 - The *sum of two positive numbers* exceeds the maximum positive value that can be represented using n bits: $2^{n-1} - 1$
 - The *sum of two negative numbers* falls below the minimum negative value that can be represented using n bits: -2^{n-1}

Example

- Four-bit arithmetic:
 - Sixteen possible values
 - Positive overflow happens when result > 7
 - Negative overflow happens when result < -8
- Eight-bit arithmetic:
 - 256 possible values
 - Positive overflow happens when result > 127
 - Negative overflow happens when result < -128

Overflow handling (III)

- MIPS (Million Instructions per second) architecture handles signed and unsigned overflows in a very different fashion:

- **Ignores unsigned overflows**
 - Implements **modulo 2^n arithmetic**
- Generates an **interrupt** whenever it detects a **signed overflows**
 - Lets the OS handle the condition

1111 (4 bits)
Overflow in 3 bit representation
Perform modulo 2
Result will be 111
(no overflow in 3 bit)

Example: Adding two positive numbers

Adding 5 + 3 should equal 8, which is outside the range of -8 to 7.

- 5 in 4-bit binary is 0101
- 3 in 4-bit binary is 0011
- 0101 (5) + 0011 (3) ----- 1000
(-8 in two's complement)

*The result is 1000, which the computer interprets as -8 (a negative number). Since two positive numbers were added to produce a negative result, an **overflow exception** is raised in MIPS.

Example: Adding two negative numbers

Adding -4 + -5 should equal -9, which is outside the range of -8 to 7.

- -4 in 4-bit binary is 1100
- -5 in 4-bit binary is 1101
- 1100 (-4) + 1101 (-5) ----- 1 0111
(7 in two's complement)

*The result in 4 bits is 0111, which the computer interprets as 7 (a positive number). Since two negative numbers were added to produce a positive result, an **overflow exception** is raised.

Subtraction

- Can be implementing by
 - Specific hardware
 - Negating the subtrahend

Negating a number

- Toggle all bits then add one

In 4-bit arithmetic (I)

0001	1	1110	+1 = 1111	-1
0010	2	1101	+1 = 1110	-2
0011	3	1100	+1 = 1101	-3
0100	4	1011	+1 = 1100	-4
0101	5	1010	+1 = 1011	-5
0110	6	1001	+1 = 1010	-6
0111	7	1000	+1 = 1001	-7

In 4-bit arithmetic (II)

1001	-7	0110	+1 = 0111	7
1010	-6	0101	+1 = 0110	6
1011	-5	0100	+1 = 0101	5
1100	-4	0011	+1 = 0100	4
1101	-3	0010	+1 = 0011	3
1110	-2	0001	+1 = 0010	2
1111	-1	0000	+1 = 0001	1

Addition and Subtraction

- Let us consider the magnitude of the two numbers as A and B.
- when the signed numbers are added or subtracted, there are eight different conditions to consider, depending on the sign of the numbers and the operation performed.
- These conditions are listed in the first column of the table below *(next slide)*.
- The other column in the table show the actual operation to be performed with the magnitude of the numbers.

Addition and Subtraction...

Operation	ADD Magnitudes	SUBTRACT Magnitudes		
		$A > B$	$A < B$	$A = B$
$(+A) + (+B)$	$+ (A + B)$			
$(+A) + (-B)$		$+ (A - B)$	$- (B - A)$	$+ (A - B)$
$(-A) + (+B)$		$- (A - B)$	$+ (B - A)$	$+ (A - B)$
$(-A) + (-B)$	$- (A + B)$			
$(+A) - (+B)$		$+ (A - B)$	$- (B - A)$	$+ (A - B)$
$(+A) - (-B)$	$+ (A + B)$			
$(-A) - (+B)$	$- (A + B)$			
$(-A) - (-B)$		$- (A - B)$	$+ (B - A)$	$+ (A - B)$

The last column is required to prevent negative zero. In other words, when two equal numbers are subtracted, the result should be +0 not -0.

Addition and Subtraction...

Addition (subtraction) algorithm:

- When the signs of A and B are identical (different), add the two magnitudes and attach the sign of A to the result.
- When the signs of A and B are different (identical), compare the magnitudes.

Addition and Subtraction...

Hardware Implementation

- To implement the two arithmetic operations with hardware, it is first required that the two numbers be stored in registers.
- Let A and B be two registers that hold the magnitude of the numbers, and A_s and B_s be two flipflops that hold the corresponding signs.
- The results of the operation may be transferred to a third register however, a saving achieved if the result is transferred into A and A_s .
- Thus A and A_s together form an Accumulator register.

In a computer's central processing unit (CPU), the accumulator is a register in which intermediate arithmetic and logic results are stored.

Addition and Subtraction...

Hardware Implementation...

- Consider now the hardware implementation of the algorithms above.
- First, a parallel adder is needed to perform the micro operation $A+B$.
- Second, comparator circuit is needed to establish if $A>B$, $A=B$, or $A<B$.
- Third, two parallel subtractor circuits are required to perform the micro operation $A-B$ and $B-A$.
- The sign relationship can be determined from an XOR gate with A_s and B_s as inputs.

Addition and Subtraction...

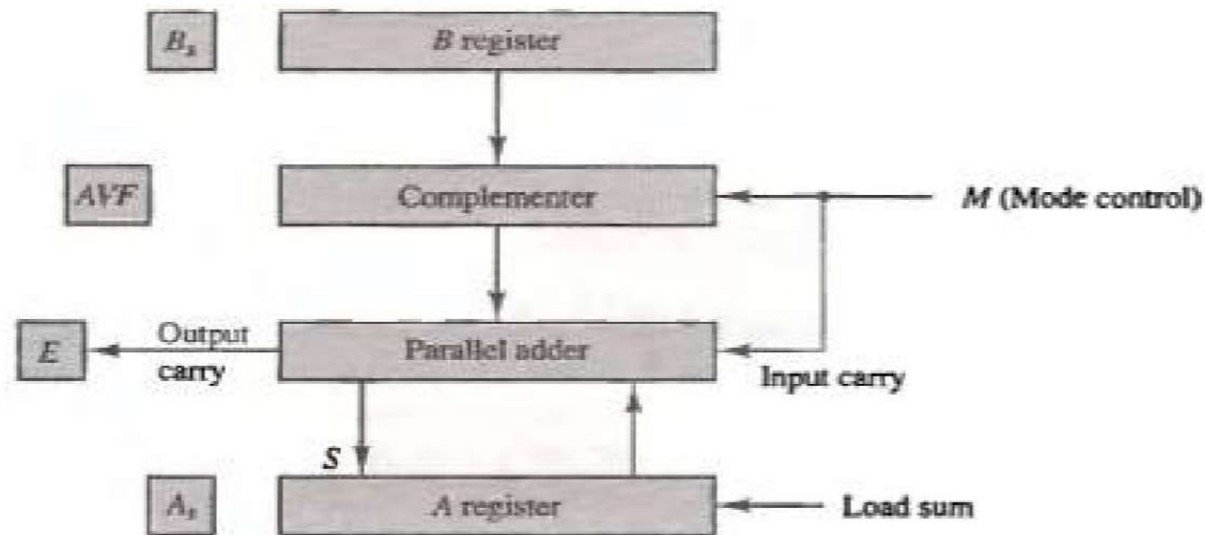
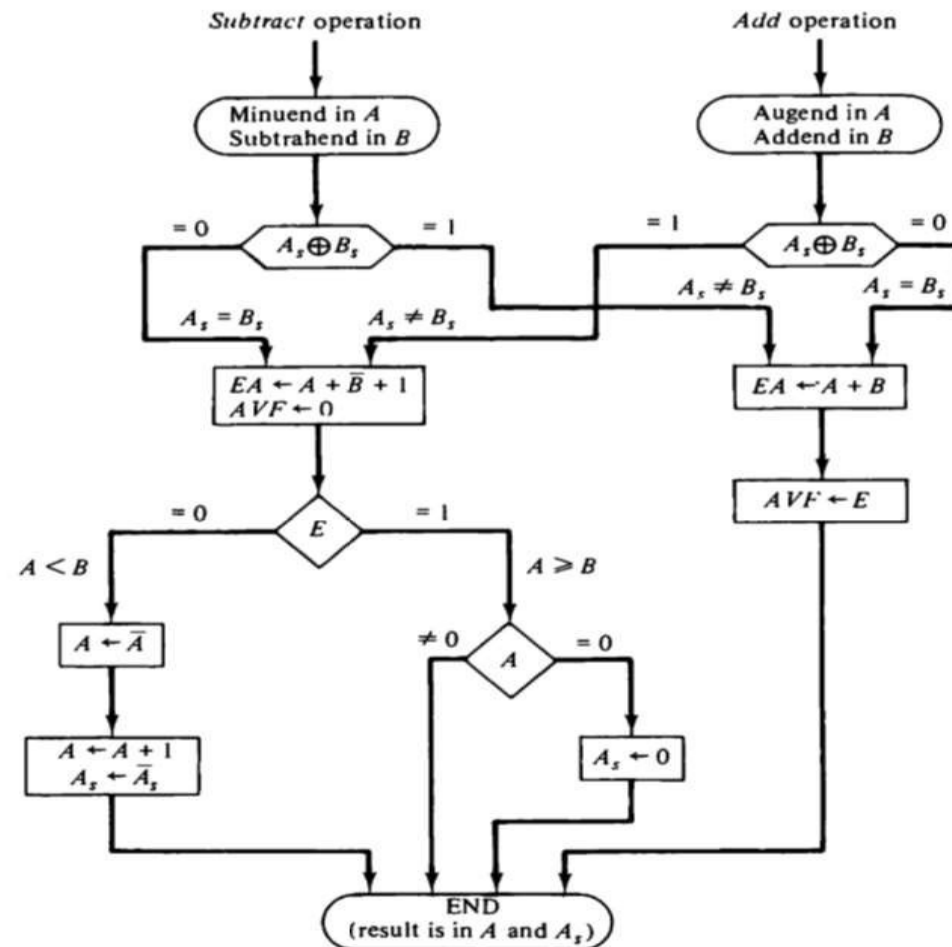


Figure 10-1 Hardware for signed-magnitude addition and subtraction.

- The output carry is transferred to flip-flop E.
- The complementor consists of exclusive-OR gates and the parallel adder consists of full adder circuit.

Addition and Subtraction...

Hardware Algorithm



Addition and Subtraction...

- The two signs A_s and B_s are compared by an exclusive-OR gate.
- For an add operation, identical signs dictate that the magnitudes be added, for subtract operation different signs dictate that the Magnitudes be added. The magnitudes are added with a micro operation $E A \leftarrow A+B$. Where $E A$ is a register that combines E and A .
- For A , 0 indicates that $A < B$, for this case it is necessary to take the 2's complement of the value in A . This operation can be done with one micro operation $A \leftarrow \bar{A}+1$.

However, we assume that A register as circuits for microoperations complement and increment, so the 2's complement is obtain from these two micro operations.

The value in AVF provides an overflow indication. The final value of E is immaterial.

Addition and Subtraction...

Addition and Subtraction with signed 2's complement data

- The left most bit of binary number represents the sign bit; 0 for positive and 1 for negative. If the sign bit is 1, the entire number is represented in 2's complement form.
- The addition of two numbers in signed-2's complement form consists of adding the number with the sign bits treated the same as the other bits of the number.
- A carry out of the sign bit position is discarded .

Addition and Subtraction...

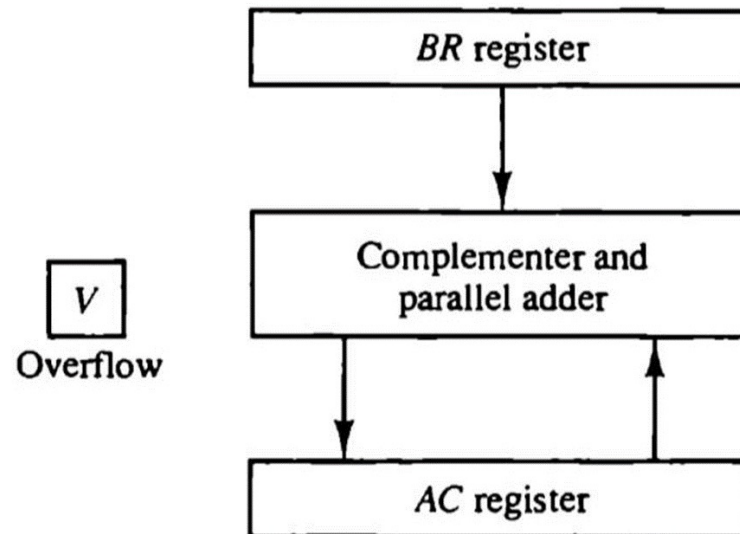
Addition and Subtraction with signed 2's complement data...

- The subtraction consists of first taking the 2's complement of the subtrahend and then adding it to the minuend.
- When two numbers of n digits each are added and the sum occupies $n+1$ Digits, we say that an overflow occurred.
- When the two carries are applied to an exclusive-OR gate, the overflow is detected when the output of the gate is equal to 1.

Addition and Subtraction...

Addition and Subtraction with signed 2's complement data...

Figure 10-3 Hardware for signed-2's complement addition and subtraction.



- The left most bit in AC and BR represents the sign bits of the numbers.
- The over flow flip-flops V is set to 1 if there is an overflow. The output carry in this case is discarded.

Addition and Subtraction...

Addition and Subtraction with signed 2's complement data...

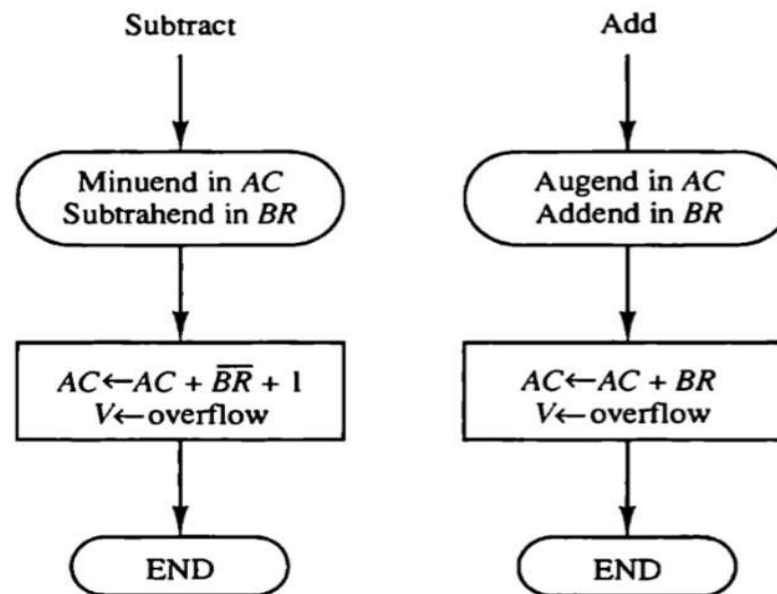


Figure 10-4 Algorithm for adding and subtracting numbers in signed-2's complement representation.

The sum is obtained by adding the contents of AC and BR (including their sign bits). The overflow bit V is set to 1 if the ex-OR of the last two carries is 1, and it is cleared to 0 otherwise.